

Task 3: Unit Testing and Logging in Product API

Java Backend Development Internship – InternSpark

Name: Karakamukkala Matam Likhitha Gowri

Technology Stack: Java, Spring Boot, Spring Data JPA, JUnit 5, Mockito, SLF4J, Logback, Maven, IntelliJ IDEA

Objective

The objective of this task is to improve the reliability and maintainability of the Product Management REST API by implementing unit testing and application logging. Unit tests were written using JUnit 5 and Mockito to verify service layer functionality, while SLF4J with Logback was integrated to generate application logs for important operations.

Technologies Used

- Java 17
- Spring Boot
- Spring Data JPA
- MySQL
- JUnit 5
- Mockito
- SLF4J
- Logback
- Maven
- IntelliJ IDEA

Features Implemented

Unit Testing

Implemented unit tests for the Product Service layer using JUnit 5 and Mockito. Repository methods were mocked using Mockito to test business logic independently without connecting to the database.

The following test cases were implemented:

- Save Product

- Get All Products
- Get Product By ID
- Update Product
- Delete Product

```

1 package com.example.productapi.service;
2 import java.util.List;
3 import com.example.productapi.entity.Product;
4 import com.example.productapi.repository.ProductRepository;
5 import org.junit.jupiter.api.Test;
6 import org.junit.jupiter.api.extension.ExtendWith;
7 import org.mockito.InjectMocks;
8 import org.mockito.Mock;
9 import org.mockito.junit.jupiter.MockitoExtension;
10
11 import java.util.Arrays;
12
13 import static org.junit.jupiter.api.Assertions.*;
14 import static org.mockito.Mockito.*;
15
16 @ExtendWith(MockitoExtension.class)
17 class ProductServiceTest {
18
19     @Mock
20     private ProductRepository productRepository;
21
22     @InjectMocks
23     private ProductService productService;
24

```

```

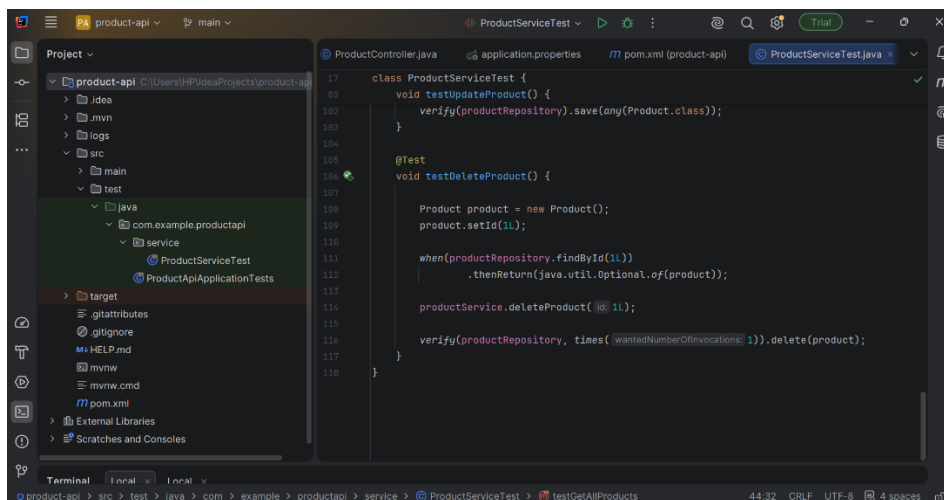
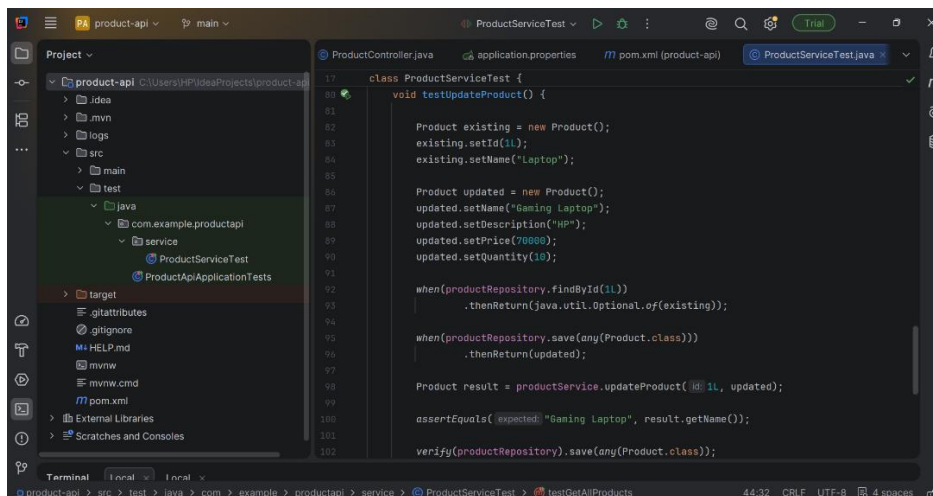
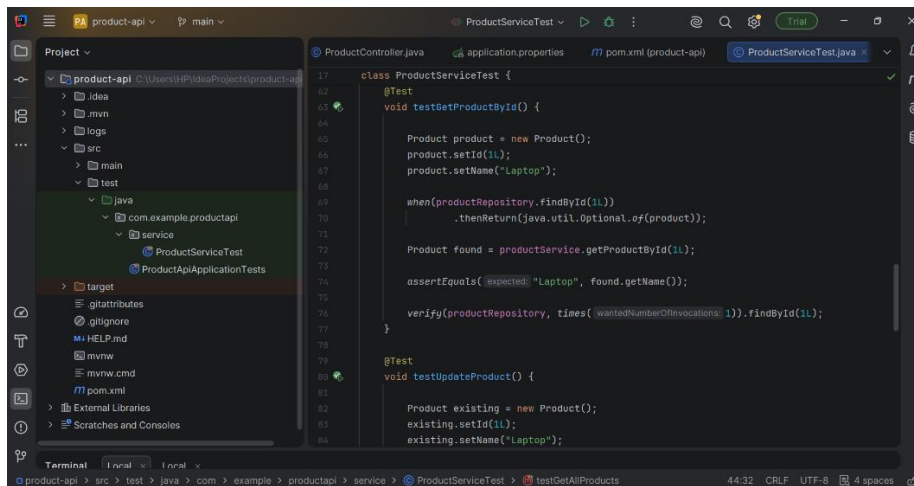
17 class ProductServiceTest {
18     private ProductService productService;
19
20     @Test
21     void testSaveProduct() {
22
23         Product product = new Product();
24         product.setName("Laptop");
25         product.setDescription("HP Laptop");
26         product.setPrice(50000);
27         product.setQuantity(5);
28
29         when(productRepository.save(product)).thenReturn(product);
30
31         Product savedProduct = productService.saveProduct(product);
32
33         assertEquals("expected: 'Laptop', savedProduct.getName());
34
35         verify(productRepository, times(wantedNumberOfInvocations: 1)).save(product);
36     }
37
38     @Test
39     void testGetAllProducts() {
40

```

```

17 class ProductServiceTest {
18     void testGetAllProducts() {
19
20         Product p1 = new Product();
21         p1.setName("Laptop");
22
23         Product p2 = new Product();
24         p2.setName("Phone");
25
26         when(productRepository.findAll())
27             .thenReturn(Arrays.asList(p1, p2));
28
29         List<Product> products = productService.getAllProducts();
30
31         assertEquals("expected: 2, products.size()");
32
33         verify(productRepository, times(wantedNumberOfInvocations: 1)).findAll();
34     }
35
36     @Test
37     void testGetProductById() {
38
39         Product product = new Product();
40         product.setId(1L);

```



Application Logging

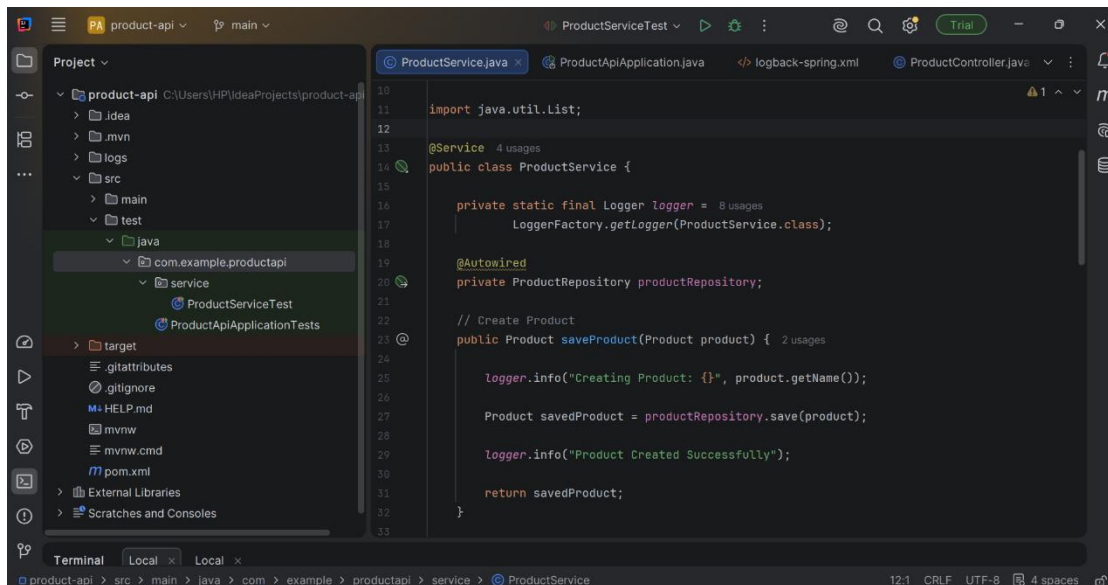
Implemented application logging using SLF4J Logger and Logback.

Logging statements were added for the following operations:

- Product Creation

- Fetching Products
- Updating Products
- Deleting Products
- Successful Completion of Operations

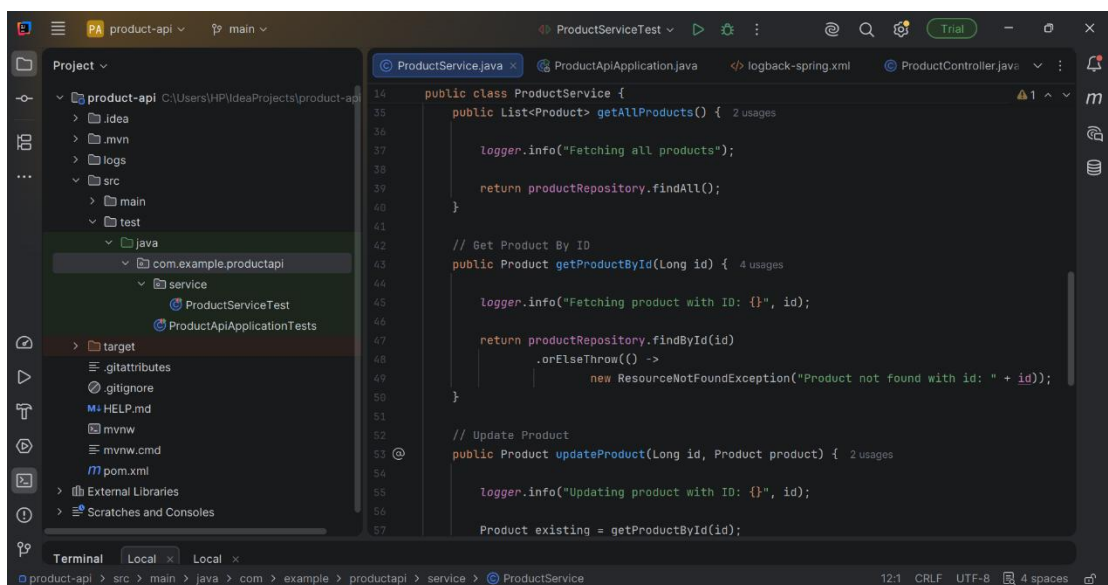
This helps monitor application behavior and simplifies debugging.



```

10
11 import java.util.List;
12
13 @Service 4 usages
14 public class ProductService {
15
16     private static final Logger logger = 8 usages
17         LoggerFactory.getLogger(ProductService.class);
18
19     @Autowired
20     private ProductRepository productRepository;
21
22     // Create Product
23     public Product saveProduct(Product product) { 2 usages
24
25         logger.info("Creating Product: {}", product.getName());
26
27         Product savedProduct = productRepository.save(product);
28
29         logger.info("Product Created Successfully");
30
31         return savedProduct;
32     }
33

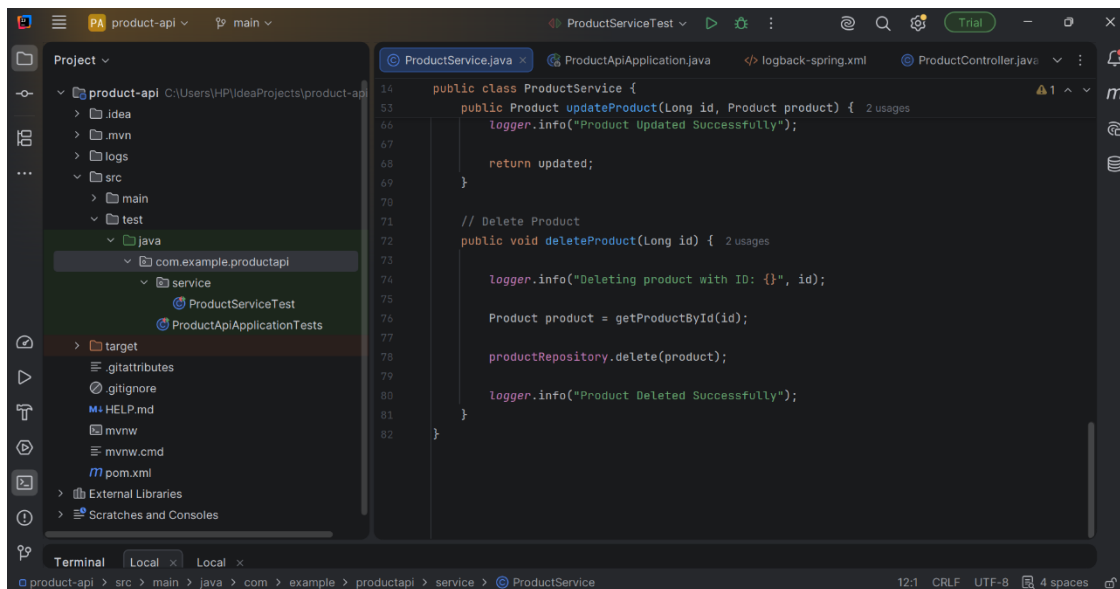
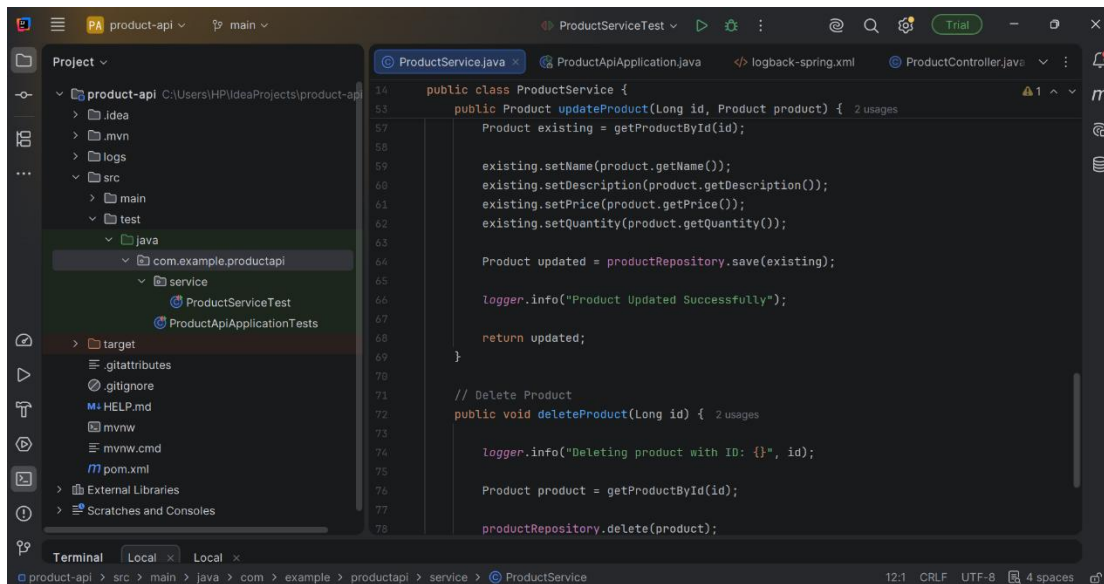
```



```

14 public class ProductService {
15
16     public List<Product> getAllProducts() { 2 usages
17
18         logger.info("Fetching all products");
19
20         return productRepository.findAll();
21     }
22
23     // Get Product By ID
24     public Product getProductById(Long id) { 4 usages
25
26         logger.info("Fetching product with ID: {}", id);
27
28         return productRepository.findById(id)
29             .orElseThrow(() ->
30                 new ResourceNotFoundException("Product not found with id: " + id));
31     }
32
33     // Update Product
34     public Product updateProduct(Long id, Product product) { 2 usages
35
36         logger.info("Updating product with ID: {}", id);
37
38         Product existing = getProductById(id);
39

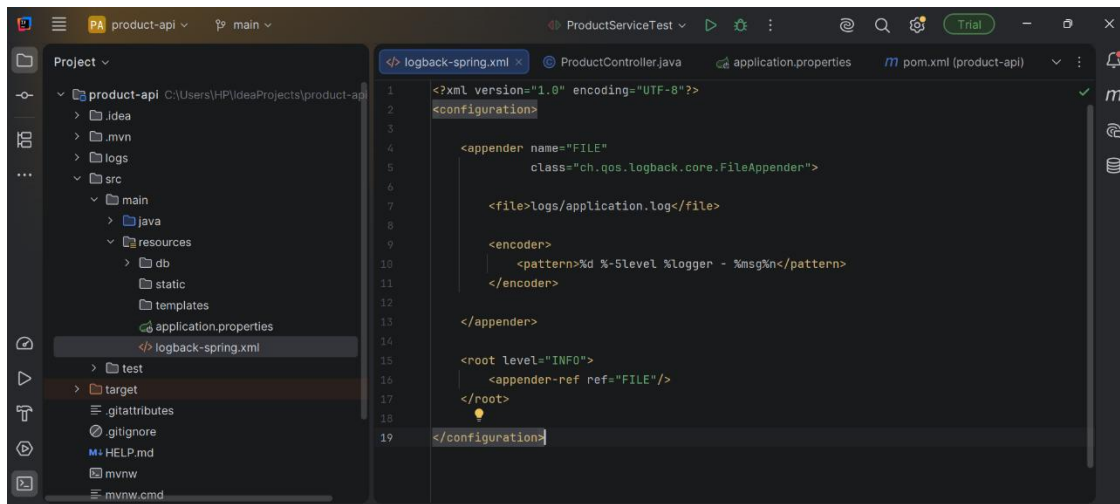
```



Logback Configuration

Configured Logback using the `logback-spring.xml` file.

The configuration records application logs and stores them in the `application.log` file while also displaying logs in the console.

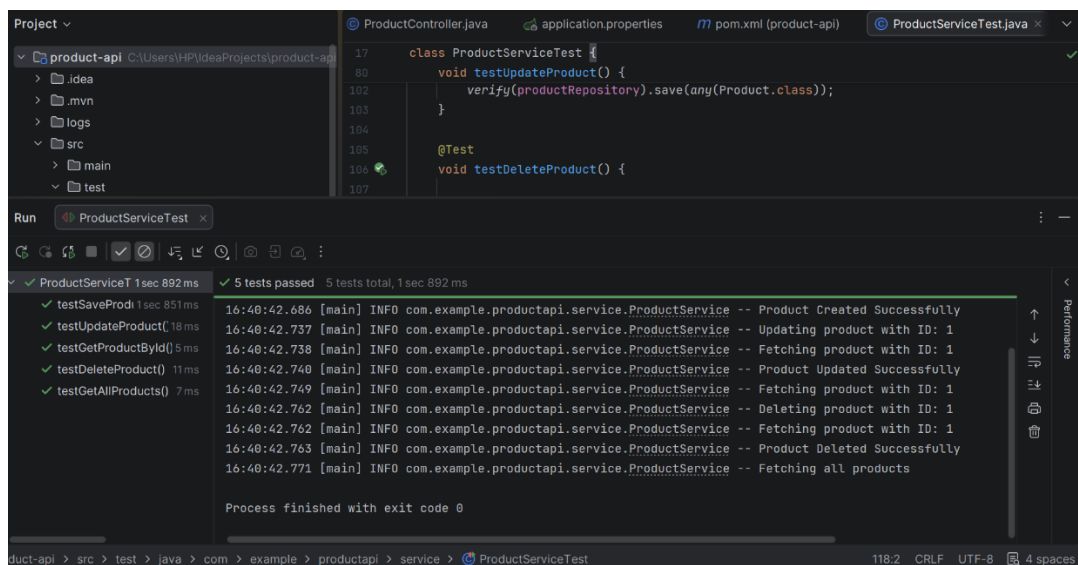


Console Output

After executing the application and unit tests, the console displays INFO logs for every major operation.

Example operations logged include:

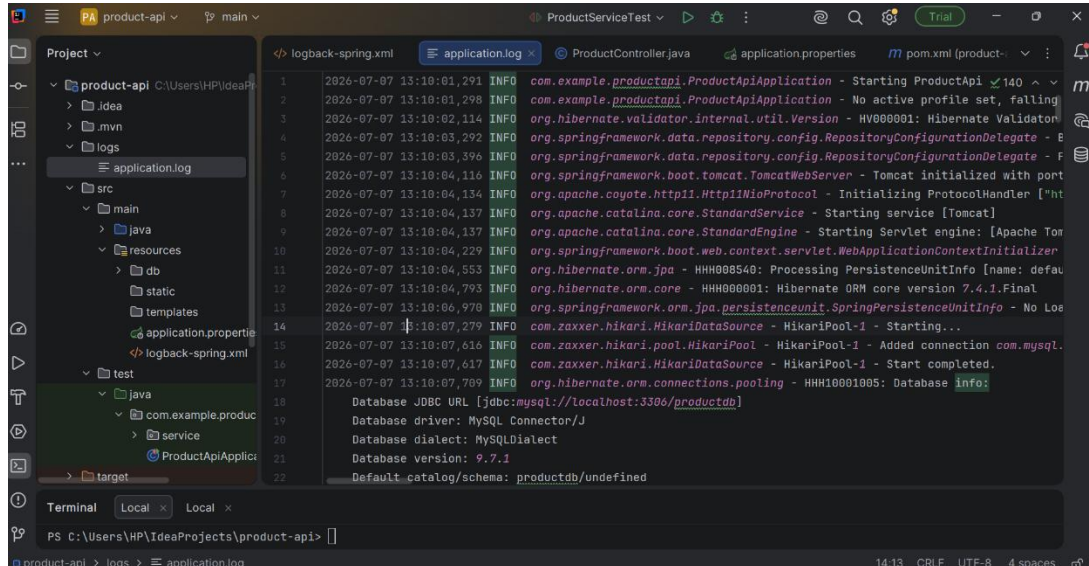
- Creating Product
- Product Created Successfully
- Fetching Product
- Updating Product
- Product Updated Successfully
- Deleting Product
- Product Deleted Successfully



Application Log File

All application logs are stored inside the logs/application.log file.

This log file helps monitor API execution and troubleshoot issues efficiently.



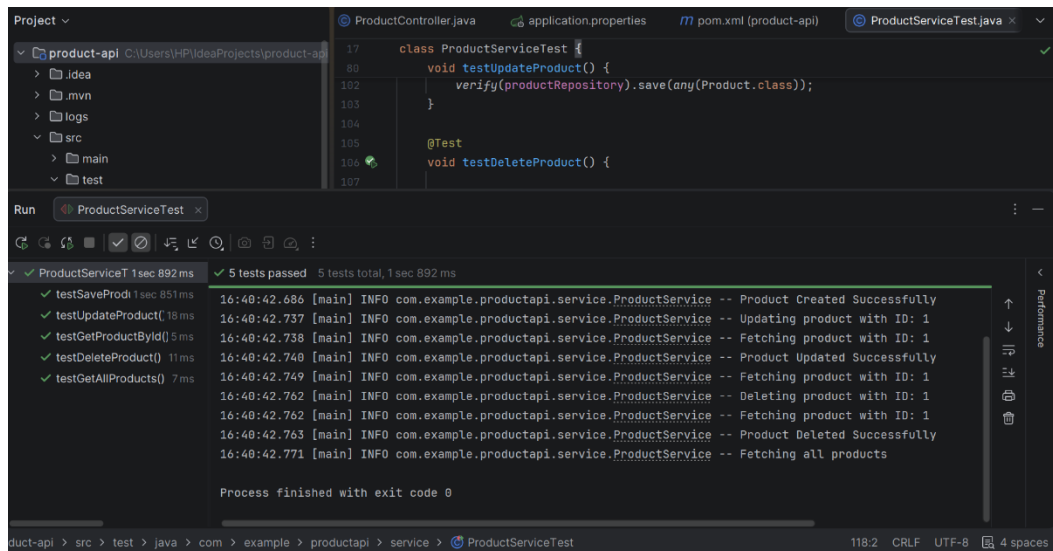
```
1 2026-07-07 13:10:01,291 INFO com.example.productapi.ProductApiApplication - Starting ProductApiApplication
2 2026-07-07 13:10:01,298 INFO com.example.productapi.ProductApiApplication - No active profile set, falling
3 2026-07-07 13:10:02,114 INFO org.hibernate.validator.internal.util.Version - HV000001: Hibernate Validator
4 2026-07-07 13:10:03,292 INFO org.springframework.data.repository.config.RepositoryConfigurationDelegate - E
5 2026-07-07 13:10:03,396 INFO org.springframework.data.repository.config.RepositoryConfigurationDelegate - F
6 2026-07-07 13:10:04,116 INFO org.springframework.boot.tomcat.TomcatWebServer - Tomcat initialized with port
7 2026-07-07 13:10:04,134 INFO org.apache.coyote.http11.Http11NioProtocol - Initializing ProtocolHandler ["ht
8 2026-07-07 13:10:04,137 INFO org.apache.catalina.core.StandardService - Starting service [Tomcat]
9 2026-07-07 13:10:04,137 INFO org.apache.catalina.core.StandardEngine - Starting Servlet engine: [Apache Tom
10 2026-07-07 13:10:04,229 INFO org.springframework.boot.web.context.servlet.WebApplicationContextInitializer
11 2026-07-07 13:10:04,553 INFO org.hibernate.orm.jpa - HH0000540: Processing PersistenceUnitInfo [name: defau
12 2026-07-07 13:10:04,793 INFO org.hibernate.orm.core - HH000001: Hibernate ORM core version 7.4.1.Final
13 2026-07-07 13:10:06,970 INFO org.springframework.orm.jpa.persistenceunit.SpringPersistenceUnitInfo - No Loc
14 2026-07-07 13:10:07,279 INFO com.zaxxer.hikari.HikariDataSource - HikariPool-1 - Starting...
15 2026-07-07 13:10:07,616 INFO com.zaxxer.hikari.pool.HikariPool - HikariPool-1 - Added connection com.mysql
16 2026-07-07 13:10:07,617 INFO com.zaxxer.hikari.HikariDataSource - HikariPool-1 - Start completed.
17 2026-07-07 13:10:07,709 INFO org.hibernate.orm.connections.pooling - HH000005: Database info:
    Database JDBC URL [jdbc:mysql://localhost:3306/productdb]
    Database driver: MySQL Connector/J
    Database dialect: MySQLDialect
    Database version: 9.7.1
    Default catalog/schema: productdb/undefined
```

Unit Test Execution

All unit tests were executed successfully using JUnit 5.

The following test cases passed successfully:

- testSaveProduct()
- testGetAllProducts()
- testGetProductById()
- testUpdateProduct()
- testDeleteProduct()
- **Result: 5 Tests Passed Successfully**



```
Project ▾
  product-api C:\Users\HP\IdeaProjects\product-api
    .idea
    .mvn
    logs
    src
      main
      test

ProductController.java application.properties pom.xml (product-api) ProductServiceTest.java x ▾
17 class ProductServiceTest {
89 void testUpdateProduct() {
102     verify(productRepository).save(any(Product.class));
103 }
104
105 @Test
106 void testDeleteProduct() {
107

Run ProductServiceTest x ▾
16:40:42.686 [main] INFO com.example.productapi.service.ProductService -- Product Created Successfully
16:40:42.737 [main] INFO com.example.productapi.service.ProductService -- Updating product with ID: 1
16:40:42.738 [main] INFO com.example.productapi.service.ProductService -- Fetching product with ID: 1
16:40:42.740 [main] INFO com.example.productapi.service.ProductService -- Product Updated Successfully
16:40:42.749 [main] INFO com.example.productapi.service.ProductService -- Fetching product with ID: 1
16:40:42.762 [main] INFO com.example.productapi.service.ProductService -- Deleting product with ID: 1
16:40:42.763 [main] INFO com.example.productapi.service.ProductService -- Fetching product with ID: 1
16:40:42.771 [main] INFO com.example.productapi.service.ProductService -- Product Deleted Successfully
16:40:42.771 [main] INFO com.example.productapi.service.ProductService -- Fetching all products

Process finished with exit code 0

product-api > src > test > java > com > example > productapi > service > ProductServiceTest 118:2 CRLF UTF-8 4 spaces
```

Outcome

Successfully implemented unit testing and logging in the Product Management REST API.

The application is now capable of:

- Performing automated unit testing using JUnit 5 and Mockito.
- Recording application events using SLF4J and Logback.
- Improving code quality through automated verification.
- Simplifying debugging and maintenance using detailed application logs.

This task enhanced the reliability, maintainability, and overall quality of the backend application.